

Music-Drone-225 飞行控制器

数学原理与代码实现全解

many_controller / px4ctrl 模块

自动生成文档

2026 年 8 月 10 日

目录

1	系统架构概览	4
1.1	双环解耦架构	4
1.2	控制器后端选择（编译期宏）	4
2	坐标系与符号约定	4
2.1	坐标系定义	4
2.2	符号表	5
2.3	李群运算	5
3	飞行器动力学模型	5
3.1	连续时间动力学	5
3.2	一阶欧拉离散化（ZOH）	6
3.3	气动阻力修正	6
4	微分平坦性	6
4.1	推力方向重构	6
4.2	完整姿态重构	6
4.3	角速度前馈	6
5	飞行状态机 (FSM)	7
5.1	状态集合与转移条件	7
5.2	无遥控自动时序（USE_WITHOUT_RC）	7
5.3	8 字参考轨迹（set_2D8_ref）	7
6	串级 PID 控制器 (QuadControl)	7
6.1	控制链路	7
6.2	位置环（P 控制）	7
6.3	速度环（PID 控制）	8

目录	2
6.4 微分平坦姿态生成	8
6.5 姿态环（四元数 P 控制）	8
6.6 手动模式直通	8
7 SO(3) 流形线性时变 MPC (LTV-MPC / GptMpcControl)	8
7.1 系统状态与误差坐标	9
7.2 误差动力学线性化	9
7.3 预测模型凝聚（批量展开）	9
7.4 二次规划目标函数	9
7.5 输入盒约束	10
7.6 滚动时域执行	10
7.7 参考来源优先级	10
7.8 推力单位换算	10
8 非线性 MPC (SolverNmpc) ——四后端统一框架	10
8.1 统一外壳（pimpl 模式）	10
8.2 NLP 问题形式	11
8.3 通用代价函数	11
8.4 后端 3: acados（当前默认）	11
8.5 后端 2/5: Ipopt	12
8.6 后端 4: NLopt	12
8.7 四后端对比	12
9 桶滚轨迹生成 (Barrel Roll Trajectory)	12
9.1 五段结构	12
9.2 七次 Hermite 多项式（起飞/进出段）	12
9.3 滚转段解析螺旋线	13
9.4 微分平坦姿态生成	13
10 Minimum-Snap 轨迹生成	13
10.1 分段多项式表示	13
10.2 优化问题	13
11 SCP 最短时间轨迹优化 (GptTraj)	13
11.1 连续动力学	14
11.2 QP 决策变量布局	14
11.3 目标函数	14
11.4 CSTC 航点进度约束	14
11.5 SCP 迭代流程	15
11.6 航点通过时刻恢复	15
11.7 时间缩放热启动与插值	15

目录	3
12 内环：角速度控制与电机分配	15
12.1 角速度 PID	15
12.2 控制分配（逆矩阵法）	16
13 关键参数汇总	16
14 总结：控制链路数据流	16

1 系统架构概览

1.1 双环解耦架构

本项目采用外环-内环解耦结构，两个独立 ROS2 节点分别运行：

- **外环** px4ctrl_node (200 Hz): 接收位置/姿态估计，运行飞行状态机与高级控制器，输出期望角速度 ω_d 和合力 F_T (牛顿制)。
- **内环** px4ctrlrate_node (400 Hz): 接收 (ω_d, F_T) ，运行角速度 PID 并通过逆矩阵分配法输出四个电机推力指令。

1.2 控制器后端选择 (编译期宏)

外环控制器通过 px4ctrlfsm.h 中的宏在编译期选择：

```
#define PX4CTRL_PRIMARY_CONTROLLER 3
// 0=QuadControl (串级PID)
// 1=GptMpcControl (SO(3) LTV-MPC, OSQP)
// 2=SolverNmpc + Ipopt+Eigen 后端
// 3=SolverNmpc + acados 后端 (当前默认)
// 4=SolverNmpc + Nlopt+Eigen 后端
// 5=SolverNmpc + Ipopt+CasADi 后端
```

所有控制器共享统一的输入接口 ($\mathbf{x}_{\text{curr}}$, 参考轨迹) 和输出接口 (ω_d, F_T)。

2 坐标系与符号约定

2.1 坐标系定义

- **惯性系 $\mathcal{I}$ (ENU)**: 东-北-天, z 轴朝上, 原点固定于地面。
- **机体系 $\mathcal{B}$** : 原点位于质心, x 轴朝前, z 轴朝上 (与 NED 相反)。
- **旋转矩阵 $R \in \text{SO}(3)$** : 将机体系向量变换至惯性系, 即 $\mathbf{a}_I = R \mathbf{a}_B$ 。

2.2 符号表

符号	维度	含义
$\mathbf{p} \in \mathbb{R}^3$	3	位置 (惯性系, ENU)
$\mathbf{v} \in \mathbb{R}^3$	3	速度 (惯性系)
$R \in \text{SO}(3)$	3×3	姿态旋转矩阵
$\mathbf{q} \in \mathbb{R}^4$	4	姿态四元数 (q_w, q_x, q_y, q_z)
$\boldsymbol{\omega} \in \mathbb{R}^3$	3	角速度 (机体系)
$a_T \in \mathbb{R}$	1	质量归一化推力加速度 [m/s^2]
$F_T \in \mathbb{R}$	1	合力 ($F_T = ma_T$) [N]
m	—	无人机质量 (0.7311 kg)
g	—	重力加速度 (9.805 m/s^2)
$\mathbf{e}_3$	—	竖直单位向量 $(0, 0, 1)^T$
$\hat{\boldsymbol{\omega}}$	3×3	$\boldsymbol{\omega}$ 的反对称矩阵 (叉乘矩阵)
$\text{Exp}(\boldsymbol{\theta})$	—	指数映射 $\text{SO}(3)$, $\boldsymbol{\theta} \rightarrow R$
$\text{Log}(R)$	—	对数映射 $\text{SO}(3)$, $R \rightarrow \boldsymbol{\theta} \in \mathbb{R}^3$

2.3 李群运算

反对称矩阵:

$$\hat{\boldsymbol{\omega}} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}, \quad \hat{\boldsymbol{\omega}} \mathbf{a} = \boldsymbol{\omega} \times \mathbf{a} \quad (1)$$

指数映射 (Rodrigues 公式):

$$\text{Exp}(\boldsymbol{\theta}) = I + \frac{\sin \|\boldsymbol{\theta}\|}{\|\boldsymbol{\theta}\|} \hat{\boldsymbol{\theta}} + \frac{1 - \cos \|\boldsymbol{\theta}\|}{\|\boldsymbol{\theta}\|^2} \hat{\boldsymbol{\theta}}^2 \quad (2)$$

对数映射:

$$\text{Log}(R) = \frac{\phi}{2 \sin \phi} \begin{pmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{pmatrix}, \quad \phi = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right) \quad (3)$$

3 飞行器动力学模型

3.1 连续时间动力学

多旋翼飞行器在 ENU 坐标系中的刚体动力学 (忽略陀螺效应) 为:

$$\dot{\mathbf{p}} = \mathbf{v} \quad (4)$$

$$\dot{\mathbf{v}} = -g \mathbf{e}_3 + a_T R \mathbf{e}_3 \quad (5)$$

$$\dot{R} = R \hat{\boldsymbol{\omega}} \quad (6)$$

其中 $a_T R \mathbf{e}_3$ 为机体推力方向在惯性系下的加速度分量, $g \mathbf{e}_3$ 为重力加速度。

3.2 一阶欧拉离散化 (ZOH)

给定时间步长 Δt ，系统离散化为：

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k \quad (7)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t(-g \mathbf{e}_3 + a_{T,k} R_k \mathbf{e}_3) \quad (8)$$

$$R_{k+1} = R_k \text{Exp}(\Delta t \boldsymbol{\omega}_k) \quad (9)$$

3.3 气动阻力修正

实机参数中包含线性气动阻力项 (`aero` 参数组)，在速度环中加入机体系阻力补偿：

$$\mathbf{f}_{\text{drag}} = -\text{diag}(k_{dx}, k_{dy}, k_{dz}) \mathbf{v}_B, \quad \mathbf{v}_B = R^T \mathbf{v} \quad (10)$$

其中 $k_{dx} = 0.26$, $k_{dy} = 0.28$, $k_{dz} = 0.42$ (单位 kg/s)。

4 微分平坦性

多旋翼系统对输出 $(\mathbf{p}, \psi)$ (位置 + 偏航角) 具有微分平坦性，即任意光滑轨迹 $\mathbf{p}(t)$ 均可唯一确定全状态和控制输入。

4.1 推力方向重构

由期望加速度 $\mathbf{a}_d = \ddot{\mathbf{p}}_d + g \mathbf{e}_3$ 得到机体 z 轴方向：

$$\mathbf{b}_3 = \frac{\mathbf{a}_d}{\|\mathbf{a}_d\|}, \quad a_T = \|\mathbf{a}_d\| \quad (11)$$

4.2 完整姿态重构

给定偏航参考 ψ_d ，定义水平参考向量 $\mathbf{b}_{1c} = (\cos \psi_d, \sin \psi_d, 0)^T$ ，则：

$$\mathbf{b}_2 = \frac{\mathbf{b}_3 \times \mathbf{b}_{1c}}{\|\mathbf{b}_3 \times \mathbf{b}_{1c}\|}, \quad \mathbf{b}_1 = \mathbf{b}_2 \times \mathbf{b}_3, \quad R_d = \begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \mathbf{b}_3 \end{bmatrix} \quad (12)$$

4.3 角速度前馈

对姿态矩阵关于时间求导 $\dot{R}_d = R_d \widehat{\boldsymbol{\omega}}_d$ ，因此：

$$\widehat{\boldsymbol{\omega}}_d = R_d^T \dot{R}_d \implies \boldsymbol{\omega}_d = \text{vex}\left(R_d^T \dot{R}_d\right) \quad (13)$$

桶滚轨迹 (`barrel_roll_trajectory.cpp`) 即通过对 $R(t)$ 解析求导获取精确角速度前馈，避免有限差分误差。

5 飞行状态机 (FSM)

5.1 状态集合与转移条件

状态机定义于 `px4ctrlfsm.h/cpp`，共 6 个状态：

状态	触发条件	行为
<code>manual_on</code>	初始状态，数据就绪前	等待
<code>manual</code>	<code>aux1</code> 拨 UP，切入 Offboard 成功	直通 RC 指令
<code>auto_hover</code>	<code>aux2</code> 拨 MID，位置有效	悬停于当前位置
<code>cmd</code>	<code>aux2</code> 拨 UP	执行预装轨迹
<code>safe</code>	RC 超时 1.5s 或 kill 开关 UP	匀速下降 0.3m/s
<code>err</code>	传感器失效	紧急处理

5.2 无遥控自动时序 (USE_WITHOUT_RC)

编译宏 `USE_WITHOUT_RC` 开启时，系统按固定时间轴自动推进状态：

$$t \geq 0.1 \text{ s} \implies \text{切入 auto_hover}$$

$$t \geq 2.1 \text{ s} \implies \text{切入 cmd (执行轨迹)}$$

调参模式下 `hover` 延迟延长至 3.0s。

5.3 8 字参考轨迹 (set_2D8_ref)

FSM 内置参数化 8 字水平轨迹，提供解析的 $\mathbf{p}/\mathbf{v}/\mathbf{a}/\text{jerk}/\text{snap}$ ，以 Logistic 包络从 0 加速至 2.5m/s，可作为 MPC 的多步预瞄参考。

6 串级 PID 控制器 (QuadControl)

源码：`controller.h / controller.cpp`，后端编号 0。

6.1 控制链路

$$\mathbf{p}_d \xrightarrow{\text{位置 P}} \mathbf{v}_d \xrightarrow{\text{速度 PID}} \mathbf{F}_d \xrightarrow{\text{微分平坦}} R_d, a_T \xrightarrow{\text{姿态 P}} \boldsymbol{\omega}_d$$

6.2 位置环 (P 控制)

$$\mathbf{v}_d = \mathbf{v}_{\text{ff}} + K_p^{\text{pos}}(\mathbf{p}_d - \mathbf{p}) \quad (14)$$

$K_p^{\text{pos}} = \text{diag}(2.5, 1.5, 1.5)$ ， $\mathbf{v}_{\text{ff}}$ 为轨迹前馈速度。

6.3 速度环 (PID 控制)

$$\mathbf{a}_d = K_p^{vel} e_v + K_i^{vel} \int e_v dt + K_d^{vel} \dot{e}_v + \mathbf{a}_{ff} \quad (15)$$

$K_p^{vel} = \text{diag}(4.0, 3.0, 2.5)$, $K_i^{vel} = \text{diag}(0.8, 0.8, 0.5)$, $K_d^{vel} = \text{diag}(0.2, 0.2, 0.0)$ 。

加入重力补偿与气动阻力前馈:

$$\mathbf{a}_T^{\text{vec}} = \mathbf{a}_d + g \mathbf{e}_3 - R \text{diag}(k_{dx}, k_{dy}, k_{dz}) R^T \mathbf{v} \quad (16)$$

6.4 微分平坦姿态生成

由 $\mathbf{a}_T^{\text{vec}}$ 按第 5 节公式重构期望姿态 R_d 与推力加速度 $a_T = \|\mathbf{a}_T^{\text{vec}}\|$ 。总推力:

$$F_T = m a_T \quad (17)$$

6.5 姿态环 (四元数 P 控制)

将当前姿态 $\mathbf{q}$ 与期望姿态 $\mathbf{q}_d$ 的误差用四元数表示:

$$\mathbf{q}_e = \mathbf{q}^{-1} \otimes \mathbf{q}_d, \quad \text{若 } q_{e,w} < 0 \text{ 则 } \mathbf{q}_e \leftarrow -\mathbf{q}_e \quad (18)$$

(后一步消除双重覆盖歧义, 保证走最短路径。) 期望角速度:

$$\boldsymbol{\omega}_d = 2 K_q \text{sign}(q_{e,w}) \mathbf{q}_{e,\text{vec}} + \boldsymbol{\omega}_{ff} \quad (19)$$

其中姿态增益 $K_q = \text{diag}(12, 13, 2)$, $\boldsymbol{\omega}_{ff}$ 为轨迹角速度前馈。

6.6 手动模式直通

在 manual 状态下 (`fsm_state==1`), 控制器绕过位置/速度环, 直接由 RC 摇杆生成指令:

$$\mathbf{q}_e = \mathbf{q}^{-1} \otimes \mathbf{q}_{rc}, \quad (\text{若 } q_{e,w} < 0 \text{ 取反}) \quad (20)$$

$$\boldsymbol{\omega}_d = 8 \mathbf{q}_{e,\text{vec}} + (0, 0, \psi_{\text{rate}})^T \quad (21)$$

$$F_T = 4 (u_{\min} + \text{throttle} \cdot (u_{\max} - u_{\min})) \quad (22)$$

此逻辑在 QuadControl、GptMpcControl、SolverNmpc 三者中一致实现, 保证后端切换时手动行为不变。

7 $SO(3)$ 流形线性时变 MPC (LTV-MPC / GptMpcControl)

源码: `gptmpc.h` / `gptmpc.cpp`, 后端编号 1, 求解器为 OSQP。

7.1 系统状态与误差坐标

状态定义为 $\mathbf{x} = (\mathbf{p}, \mathbf{v}, R) \in \mathbb{R}^3 \times \mathbb{R}^3 \times SO(3)$, 控制输入为 $\mathbf{u} = (a_T, \boldsymbol{\omega}) \in \mathbb{R} \times \mathbb{R}^3$ 。

为在欧式空间中处理 $SO(3)$ 元素, 在参考轨迹 $(\mathbf{p}^d, \mathbf{v}^d, R^d, a_T^d, \boldsymbol{\omega}^d)$ 处定义 9 维局部误差坐标:

$$\mathbf{e} = \begin{pmatrix} \delta \mathbf{p} \\ \delta \mathbf{v} \\ \boldsymbol{\phi} \end{pmatrix} \in \mathbb{R}^9, \quad \delta \mathbf{p} = \mathbf{p} - \mathbf{p}^d, \quad \delta \mathbf{v} = \mathbf{v} - \mathbf{v}^d, \quad \boldsymbol{\phi} = \text{Log}\left((R^d)^T R\right) \quad (23)$$

其中 $\boldsymbol{\phi} \in \mathbb{R}^3$ 为从 R^d 到 R 的旋转向量 (李代数元素)。

7.2 误差动力学线性化

在参考点 $(\mathbf{x}_k^d, \mathbf{u}_k^d)$ 处对连续动力学 (4)–(6) 进行一阶线性化并离散, 得:

$$\mathbf{e}_{k+1} = A_k \mathbf{e}_k + B_k \delta \mathbf{u}_k + \mathbf{c}_k \quad (\text{M4})$$

其中 $\delta \mathbf{u}_k = \mathbf{u}_k - \mathbf{u}_k^d$, 仿射项 $\mathbf{c}_k$ 由参考轨迹的离散误差 (defect) 产生。矩阵 $A_k \in \mathbb{R}^{9 \times 9}$, $B_k \in \mathbb{R}^{9 \times 4}$ 均通过解析推导获得。

7.3 预测模型凝聚 (批量展开)

将 N (默认 8) 步误差沿时间轴递推并堆叠为全局向量:

$$\mathbf{E} = S_x \mathbf{e}_0 + S_u \Delta \mathbf{U} + \mathbf{s}_{\text{aff}} \quad (\text{M5})$$

其中

$$\mathbf{E} = \begin{pmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_N \end{pmatrix} \in \mathbb{R}^{9N}, \quad \Delta \mathbf{U} = \begin{pmatrix} \delta \mathbf{u}_0 \\ \vdots \\ \delta \mathbf{u}_{N-1} \end{pmatrix} \in \mathbb{R}^{4N}$$

矩阵 $S_x \in \mathbb{R}^{9N \times 9}$, $S_u \in \mathbb{R}^{9N \times 4N}$ 由 A_k, B_k 递推构成下三角结构, $\mathbf{s}_{\text{aff}}$ 含各步仿射项的累积。

凝聚策略将决策变量规模由 $13N$ (状态 + 控制分离法) 压缩为 $4N$ ($N=8$ 时仅 32 维 QP)。

7.4 二次规划目标函数

定义加权预测误差代价与控制增量代价:

$$J = \mathbf{E}^T \bar{Q} \mathbf{E} + \Delta \mathbf{U}^T \bar{R} \Delta \mathbf{U} \quad (\text{M6})$$

代入 (M5) 展开, 得到标准 QP 形式:

$$\min_{\Delta \mathbf{U}} \frac{1}{2} \Delta \mathbf{U}^T H \Delta \mathbf{U} + \mathbf{q}^T \Delta \mathbf{U} \quad (\text{M8})$$

$$H = 2(S_u^T \bar{Q} S_u + \bar{R}), \quad \mathbf{q} = 2 S_u^T \bar{Q} (S_x \mathbf{e}_0 + \mathbf{s}_{\text{aff}})$$

加权矩阵 (每步相同):

$$Q = \text{diag}(15000, 15000, 15000, 40, 40, 40, 80, 80, 80) \quad (\text{位置/速度/姿态})$$

$$R = \text{diag}(0.5, 0.6, 0.6, 0.6) \quad (\text{推力加速度/三轴角速度})$$

7.5 输入盒约束

$$\mathbf{u}_{\min} - \mathbf{u}^d \leq \delta \mathbf{u} \leq \mathbf{u}_{\max} - \mathbf{u}^d \quad (\text{M7})$$

控制上下界来自 `motor` 参数组, $u_{\min}/u_{\max}$ 为单电机推力边界的归一化值。

7.6 滚动时域执行

OSQP 求得最优增量序列 $\Delta \mathbf{U}^*$ 后, 只取第一步:

$$\mathbf{u}_{\text{cmd}} = \mathbf{u}_0^d + \delta \mathbf{u}_0^* \quad (\text{M9})$$

下一控制周期平移参考轨迹, 重新线性化并求解新 QP。

7.7 参考来源优先级

`calculateControl()` 内部按以下优先级选取预瞄参考:

1. 离线 `GptTrajectoryResult` 轨迹 (`setTrajectory()` 装载后), 提供完整的 $p/v/R/a_T/\omega$ 序列;
2. 外部多点预瞄 `preview_` (`setReferencePreview()` 设置后);
3. 单个 `Ref_State_t` 的 Taylor 短时外推 (最低精度, 仅用于悬停)。

7.8 推力单位换算

MPC 输出质量归一化推力加速度 a_T , 最终乘以质量换算为牛顿制合力:

$$F_T = m a_T \quad (\text{M10})$$

8 非线性 MPC (SolverNmpc) ——四后端统一框架

源码: `solver_nmpc.h`、`solver_nmpc_common.cpp`、`solver_nmpc_backend.h`, 以及四个后端实现文件。后端编号 2/3/4/5。

8.1 统一外壳 (pimpl 模式)

`SolverNmpcControl` 通过 `pimpl` 隐藏后端实现, 内部 `Impl` 负责与后端无关的公共逻辑:

- 构造参考向量序列 (与 `GptMpcControl` 相同的三级优先策略);
- 四元数符号对齐, 消除 $\mathbf{q}/-\mathbf{q}$ 双重覆盖歧义;
- 求解失败时保持上一条有效命令 (安全回退);
- 将 a_T 换算为 $F_T = m a_T$ 。

四个后端 (`acados` / `Ipopt+Eigen` / `NLopt+Eigen` / `Ipopt+CasADi`) 均继承 `SolverNmpcBackendBase` 抽象基类, 实现统一的 `solve()` 接口。

8.2 NLP 问题形式

状态向量采用四元数表示 (10 维), 控制 4 维:

$$\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{q}) \in \mathbb{R}^{10}, \quad \mathbf{u} = (a_T, \omega_x, \omega_y, \omega_z) \in \mathbb{R}^4 \quad (24)$$

优化问题为标准多重打靶 OCP:

$$\min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, \mathbf{x}_k^{\text{ref}}) + \ell_N(\mathbf{x}_N, \mathbf{x}_N^{\text{ref}}) \quad (25)$$

$$\text{s.t.} \quad \mathbf{x}_0 = \mathbf{x}_{\text{curr}} \quad (26)$$

$$\mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1 \quad (27)$$

$$\mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max} \quad (28)$$

其中四元数动力学离散采用 $\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \exp(\frac{1}{2}\Delta t \boldsymbol{\omega}_k)$, 随后归一化。

8.3 通用代价函数

`solver_nmpc_common.cpp` 中 `solverNmpcObjective()` 定义的分阶段代价 (Eigen 后端使用; `acados` 通过生成代码内联同权重):

$$\begin{aligned} \ell = & 18 \|\delta \mathbf{p}\|^2 + 3 \|\delta \mathbf{v}\|^2 + 5 \left(1 - (\mathbf{q} \cdot \mathbf{q}^{\text{ref}})^2\right) \\ & + 0.10 (a_T - a_T^{\text{ref}})^2 + 0.12 \|\boldsymbol{\omega} - \boldsymbol{\omega}^{\text{ref}}\|^2 \\ & + 0.02 (\Delta a_T)^2 + 0.12 \|\Delta \boldsymbol{\omega}\|^2 \end{aligned} \quad (29)$$

$$\ell_N = 35 \|\delta \mathbf{p}\|^2 + 6 \|\delta \mathbf{v}\|^2 + 10 \left(1 - (\mathbf{q} \cdot \mathbf{q}^{\text{ref}})^2\right) \quad (30)$$

姿态误差项 $1 - (\mathbf{q} \cdot \mathbf{q}^{\text{ref}})^2$ 是四元数点积的自然测度, 对 $\pm \mathbf{q}$ 对称, 天然处理双重覆盖问题; $\Delta a_T, \Delta \boldsymbol{\omega}$ 为相邻控制增量的平滑正则。

8.4 后端 3: `acados` (当前默认)

源码: `solver_nmpc_acados.cpp`, 配合 `3rdpart/acados/generated/px4ctrl_nmpc/` 中脚本 `generate_px4ctrl_acados_nmpc.py` 自动生成的 C 代码。

- 求解算法: 实时迭代 SQP (RTI), QP 子问题用 HPIPM (内点法) + BLASFEO 高性能线性代数;
- 预测窗: $N = 12$ 步, $\Delta t = 0.04\text{s}$, 即 0.48s 前视窗;
- 热启动: 将上一解按时间平移作为初值 (`shift`), 否则冷启动;
- 每周设置初始状态约束 $\mathbf{x}_0$ 、各阶段输入约束和代价参考, 调用 `acados_solve()`。

`acados` 是四后端中实时性能最好的, 适合嵌入式实机运行。

8.5 后端 2/5: Ipopt

- **Ipopt + Eigen** (编号 2, `solver_nmpc_ipopt_eigen.cpp`): 手写 Eigen 雅可比/梯度, 接入 COIN-OR Ipopt (内点法), MUMPS 稀疏线性求解器;
- **Ipopt + CasADi** (编号 5, `solver_nmpc_casadi.cpp`): 用 CasADi 符号自动微分生成精确梯度/Hessian, 再交 Ipopt 求解。通过 `solver_nmpc_casadi_bridge` (纯 C 桥接) 隔离 CasADi 的 C++ 类型, 避免与主工程 ABI 冲突。

Ipopt 后端精度高、收敛稳健, 但单次求解耗时较长, 主要用于离线对比与验证。

8.6 后端 4: NLopt

源码: `solver_nmpc_nlopt_eigen.cpp`。使用 NLopt 的 SLSQP/增广拉格朗日等梯度算法, Eigen 提供目标与约束的解析梯度。作为无需外部大型依赖的轻量非线性求解备选。

8.7 四后端对比

编号	后端	求解算法	微分方式	定位
2	Ipopt+Eigen	内点法	手写解析	高精度基准
3	acados	RTI-SQP+HPIPM	生成代码	实机实时 (默认)
4	NLopt+Eigen	SLSQP	手写解析	轻量备选
5	Ipopt+CasADi	内点法	自动微分	验证/对拍

9 桶滚轨迹生成 (Barrel Roll Trajectory)

源码: `barrel_roll_trajectory.h` / `barrel_roll_trajectory.cpp`, 编译期宏 `PX4CTRL_USE_BARREL_ROLL` 时生效。

9.1 五段结构

阶段	名称	时长	描述
1	起飞	2.5 s	竖直上升 1 m, 七次 Hermite 多项式
2	稳定悬停	0.5 s	原地静止
3	进入段	1.5 s	从静止加速至轴向速度
4	滚转段	3.5 s	解析螺旋线, 七阶时间律
5	退出段	1.5 s	从轴向速度减速至静止

9.2 七次 Hermite 多项式 (起飞/进出段)

设段归一化时间 $s = t/T_{\text{seg}} \in [0, 1]$, 七阶平滑时间律为:

$$\sigma(s) = 35s^4 - 84s^5 + 70s^6 - 20s^7 \quad (31)$$

满足 $\sigma(0) = 0$, $\sigma(1) = 1$, 且端点处一至三阶导数均为零, 保证进/出段的速度、加速度、jerk 连续。

位置插值 $\mathbf{p}(t) = \mathbf{p}_{\text{start}} + (\mathbf{p}_{\text{end}} - \mathbf{p}_{\text{start}}) \sigma(s)$, 速度/加速度/jerk/snap 均通过对 σ 解析求导获得。

9.3 滚转段解析螺旋线

设螺旋轴方向为 $\hat{\mathbf{a}}$ (通常为前飞方向 $\mathbf{e}_x$), 滚转角 $\theta(t) = 2\pi \cdot \sigma(s)$ (完整一圈), 螺旋半径 r , 轴向推进速度 v_{axial} :

$$\mathbf{p}(t) = \mathbf{p}_{\text{entry}} + v_{\text{axial}} t \hat{\mathbf{a}} + r \cos \theta(t) \hat{\mathbf{n}}_1 + r \sin \theta(t) \hat{\mathbf{n}}_2 \quad (32)$$

其中 $\hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2$ 为与 $\hat{\mathbf{a}}$ 正交的基向量。对上式解析求导得到 $\mathbf{v}, \mathbf{a}, \text{jerk}, \text{snap}$ 。

9.4 微分平坦姿态生成

每个采样点由 $\mathbf{a} + g\mathbf{e}_3$ 计算推力方向 $\mathbf{b}_3$, 通过第 5 节公式重构完整姿态矩阵 $R = [\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3]$ 。角速度通过 $\hat{\boldsymbol{\omega}} = R^T \dot{R}$ (解析求导, 非差分) 精确计算, 保证 MPC 参考输入的动力学一致性, 避免有限差分误差。

10 Minimum-Snap 轨迹生成

源码: `minimum_snap_trajectory.h / minimum_snap_trajectory.cpp`, 编译期宏 `PX4CTRL_USE_BARREL_` 时生效。

10.1 分段多项式表示

给定 M 个航点, 轨迹分为 $M - 1$ 段, 每段用 n 次 (通常 $n = 7$) 多项式表示:

$$p_j(t) = \sum_{i=0}^n c_{j,i} \left(\frac{t - t_{j-1}}{T_j} \right)^i, \quad t \in [t_{j-1}, t_j] \quad (33)$$

其中 $T_j = t_j - t_{j-1}$ 为第 j 段时长, $c_{j,i}$ 为待求系数。

10.2 优化问题

最小化全局 snap (四阶导数) 的 L^2 范数:

$$\min_{\{c_{j,i}\}} \sum_{j=1}^{M-1} \int_{t_{j-1}}^{t_j} \left\| \frac{d^4 p_j}{dt^4} \right\|^2 dt \quad (34)$$

约束条件: 航点位置约束 $p_j(t_{j-1}) = \mathbf{p}_{j-1}$, $p_j(t_j) = \mathbf{p}_j$; 内部节点到 3 阶导数连续; 端点高阶导数为零。构成带等式约束的 QP, 通过 block-diagonal 结构解析求解。

11 SCP 最短时间轨迹优化 (GptTraj)

源码: `gpttraj.h / gpttraj.cpp`, 求解器为 OSQP。实现序贯凸规划 (SCP) + 二分最短时间搜索 + CSTC 航点进度约束。

11.1 连续动力学

状态 $\mathbf{x}_k = (\mathbf{p}_k, \mathbf{v}_k, R_k)$, 控制 $\mathbf{u}_k = (a_{T,k}, \boldsymbol{\omega}_k)$, ZOH 离散同第 3 节, 时间节点数 $N = 60$ 。

11.2 QP 决策变量布局

设航点数 W :

$$\mathbf{z} = \left[\underbrace{\delta \mathbf{x}_{0:N}}_{9(N+1)}, \underbrace{\delta \mathbf{u}_{0:N-1}}_{4N}, \underbrace{\boldsymbol{\nu}_{0:N-1}}_{9N}, \underbrace{\boldsymbol{\lambda}}_{(N+1)W}, \underbrace{\boldsymbol{\mu}}_{NW}, \underbrace{\mathbf{s}_{\text{wp}}}_{NW}, \underbrace{\mathbf{s}_{\text{ord}}}_{N(W-1)} \right] \quad (35)$$

其中 $\boldsymbol{\nu}$ 为动力学虚拟控制量, $\lambda_{k,j}$ 为第 j 个航点在时刻 k 的剩余进度, $\mu_{k,j}$ 为单步进度消耗, $\mathbf{s}$ 为松弛变量。

11.3 目标函数

$$\begin{aligned} J = & \rho_x \sum_k \|\delta \mathbf{x}_k\|^2 + \rho_u \sum_k \|\bar{\mathbf{u}}_k - \mathbf{u}_{\text{hov}}\|^2 + \rho_{\Delta u} \sum_k \|\mathbf{u}_k - \mathbf{u}_{k-1}\|^2 \\ & + \rho_\nu \sum_k \|\boldsymbol{\nu}_k\|^2 + \rho_\mu \sum_k \|\delta \boldsymbol{\mu}_k\|^2 + w_s \sum_{k,j} (s_{\text{wp},k,j} + s_{\text{ord},k,j}) \end{aligned} \quad (36)$$

参数: $\rho_x = 1.0$, $\rho_u = 0.05$, $\rho_{\Delta u} = 0.2$, $\rho_\nu = 10^5$, $\rho_\mu = 1.0$, $w_s = 2 \times 10^5$, $\mathbf{u}_{\text{hov}} = (g, 0, 0, 0)^T$ 。

11.4 CSTC 航点进度约束

CSTC (Continuous Space-Time Constraint) 将航点次序与通过时刻统一建模。进度变量满足:

$$\lambda_{k+1,j} - \lambda_{k,j} + \mu_{k,j} = 0 \quad (\text{进度动力学}) \quad (\text{F7})$$

$$\lambda_{0,j} = 1, \quad \lambda_{N,j} = 0 \quad (\text{边界}) \quad (37)$$

$$\lambda_{k,j} \leq \lambda_{k,j+1} \quad (\text{弱次序}) \quad (38)$$

$$\mu_{k,j} \left(\|\mathbf{p}_k - \mathbf{p}_j\|^2 - r_j^2 \right) \leq \sigma_i + s_{\text{wp}} \quad (\text{空间互补}) \quad (\text{F8})$$

$$\lambda_{k,j} \mu_{k,j+1} \leq \sigma_i + s_{\text{ord}} \quad (\text{次序互补}) \quad (\text{F9})$$

松弛 $\sigma_i = \sigma_0 \cdot \text{decay}^i$ (初值 0.05, 衰减 0.5) 随 SCP 迭代几何衰减至 0, 逐步收紧约束。

11.5 SCP 迭代流程

Algorithm 1 SCP 最短时间轨迹优化

```

1: 初始化时间  $T_0$ , 用直线插值 + 速度估计初始化  $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ 
2: for 外层: 二分时间搜索 (最多 16 次) do
3:   for 内层: SCP 迭代 (最多 12 次) do
4:     在  $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$  处计算线性化  $A_k, B_k$  (前向差分  $\epsilon = 10^{-6}$ )
5:     构造并用 OSQP 求解凸 QP (含 CSTC 线性化约束)
6:     回溯线搜索 ( $\alpha \in \{1, 1/2, \dots\}$ ) 计算 merit, 接受/拒绝步
7:     自适应信赖域缩放: 接受  $\rightarrow$  扩大, 拒绝  $\rightarrow$  缩小
8:     if 动力学残差  $< 5 \times 10^{-2}$  且 CSTC 残差  $< 10^{-4}$  then
9:       break
10:    end if
11:  end for
12:  更新可行/不可行时间界, 二分  $T \leftarrow (T_{\text{feas}} + T_{\text{infeas}})/2$ 
13: end for
14: return GptTrajectoryResult ( $p/v/R/a_T/\omega$  序列)
  
```

11.6 航点通过时刻恢复

由进度变量恢复第 j 个航点的通过时刻:

$$t_j = \frac{T}{N} \sum_{k=0}^{N-1} k \mu_{k,j} \quad (\text{F13})$$

11.7 时间缩放热启动与插值

新旧时间之间位置路径不变, 速度按比例缩放 $\mathbf{v}_k^{\text{new}} = \frac{T_{\text{old}}}{T_{\text{new}}} \mathbf{v}_k^{\text{old}}$, CSTC 进度变量直接复制。查询时位置/速度/推力/角速度线性插值, 姿态用四元数 SLERP: $\mathbf{q}(t) = \text{Slerp}(\mathbf{q}_i, \mathbf{q}_{i+1}, \alpha)$ 。

12 内环：角速度控制与电机分配

源码: px4ctrlrate_node.cpp, 运行频率 400 Hz。

12.1 角速度 PID

内环接收外环输出的 ω_d, F_T , 运行角速度 PID:

$$\tau = K_p^\omega (\omega_d - \omega) + K_i^\omega \int (\omega_d - \omega) dt + K_d^\omega \dot{\omega} \quad (39)$$

增益: $K_p^\omega = \text{diag}(16, 18, 4)$, $K_i^\omega = \text{diag}(3, 3, 0.5)$, $K_d^\omega = \text{diag}(0.05, 0.05, 0)$ 。角速度微分 $\dot{\omega}$ 采用滑动窗口 TVR (SuiteSparse/UMFPACK 稀疏求解) 代替简单差分, 抑制陀螺噪声。

12.2 控制分配（逆矩阵法）

X 型四旋翼，电机安装角 $\beta = 50.56^\circ$ ，臂长 $l = 0.1125 \text{ m}$ 。力/力矩到电机推力映射：

$$\begin{pmatrix} F_T \\ \tau_x \\ \tau_y \\ \tau_z \end{pmatrix} = B \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{pmatrix} \quad (40)$$

混合矩阵 B 由电机位置、转向和扭矩系数确定，电机推力通过 B^{-1} 逆矩阵直接求解（弃用旧版 OSQP 分配器）。单电机推力经二次多项式反解转速指令。

13 关键参数汇总

参数	值	说明
m	0.7311 kg	质量
g	9.805 m/s ²	重力
J_{vx}, J_{vy}, J_{vz}	2.48/2.67/4.34 $\times 10^{-3}$	主惯量 kg · m ²
l	0.1125 m	臂长
β	50.56°	电机安装角
r_p	0.06475 m	桨半径 (5.1 寸)
外环频率	200 Hz	ctrl_freq_max
内环频率	400 Hz	ratectrl_freq_max
低通截止	10 Hz	acc/gyro LPF

14 总结：控制链路数据流

环节	数据流
离线轨迹优化	gpttraj: SCP+OSQP $\rightarrow p/v/R/a_T/\omega$ 序列
轨迹装载	setTrajectory()
外环控制 (200 Hz)	LTV-MPC 或 NMPC $\rightarrow (a_T, \omega_d)$
单位换算	$F_T = m a_T$
话题传递	/rates_thrust_setpoint
内环控制 (400 Hz)	角速度 PID $\rightarrow \tau$
控制分配	$B^{-1}(F_T, \tau) \rightarrow f_{1:4}$
PX4 底层	/fmu/in/actuator_motors

编译期控制器/轨迹选择速查

```
// px4ctrlfsm.h
#define PX4CTRL_PRIMARY_CONTROLLER 3 // 0~5
#define PX4CTRL_USE_BARREL_ROLL_CMD_TRAJECTORY 1 // 1=桶滚, 0=min-snap
```



```
// input.h
#define SIMULATION      // 仿真：使用 joy_msgs
#define USE_WITHOUT_RC  // 无遥控自动时序
```